Ethical Hacking Practical — Command-First Handbook

> Command reference for the 31 experiments. Use scanning, enumeration, vulnerability testing, and exploitation commands only against your assigned lab systems or systems for which you have explicit permission. Public-service examples are for passive lookup; do not probe unrelated public targets.

How to use this file

- Commands are grouped by experiment and purpose. Run only the commands relevant to your assigned question.
- Replace items such as `<DOMAIN>` or `<TARGET_IP>` with the instructor-provided value. Angle-bracket placeholders are not usually typed literally.
- Example-only lab addresses use the documentation ranges `192.0.2.0/24`, `198.51.100.0/24`, and `203.0.113.0/24`; they are not real target addresses.
- `sudo` is used where raw sockets, packet capture, or privileged scanning may require it.
- Tool options vary by version. Check the local help output before relying on an option.
- A command being present does not prove that a service, feed, account, driver, API, or external search source is functional.

Execution environment — read before using commands

This handbook is for a **Kali Linux virtual machine plus Windows tools on the Windows host**. It is not a claim that every command has been tested on your exact installed versions.

Label / code fence • Where to run it

`bash` • Kali VM terminal, unless the nearby text explicitly says otherwise

`powershell` • Windows PowerShell; use **Run as administrator** when the step changes system services, installs drivers, or installs Sysmon

`bat` / `cmd` • Windows Command Prompt

`text` containing DNS commands • Windows Command Prompt/PowerShell if the step says Windows; otherwise follow the section's environment note

Browser / GUI steps • The machine where that application is installed (Windows for FOCA and Windows Nessus; Kali for Kali-installed applications)

Confirm the machine and tool before starting

In Kali VM:

```
cat /etc/os-release
ip -br addr
ip route
command -v nmap
nmap --version
```

In Windows PowerShell:

```
$PSVersionTable
Get-NetIPConfiguration
Get-ComputerInfo | Select-Object WindowsProductName,WindowsVersion,OsArchitecture
```

Important compatibility notes

- **VM networking:** Kali and the vulnerable target must be connected to the intended lab network. A host-only/internal network is safer for vulnerable VMs than bridged networking.
- **Tool location matters:** a Windows installation of FOCA or Nessus does not make that command available inside Kali, and Kali's packages do not automatically install Windows applications.
- **GUI tools are not CLI tools:** FOCA, Maltego, Shodan, Nessus, GVM, Burp Suite, Wireshark, Armitage, pfSense, and Robtex have GUI/browser steps in this guide. Do not type GUI instructions into a terminal.
- **Version-sensitive examples:** theHarvester sources, Sublist3r, FOCA search engines, Armitage, GVM service names, and Snort 2/3 syntax can differ by version or external service availability. Run the local help/version checks first and follow your installed release's output.
- **Not every command is universally runnable as-is:** values such as `<TARGET_IP>`, `<DOMAIN>`, `<LAB_INTERFACE>`, `<BASE_DN>`, `<FILE_PATH>`, and `<SNORT_CONF>` must be replaced with real values from your lab. Example domains and documentation IPs are examples, not live lab targets.

- **Do not paste a whole section blindly:** run one command at a time, verify the target and expected output, and save evidence for the exam.

Contents

- [Experiment 1: Penetration testing environment setup](#experiment-1)
- [Experiment 2: Sysmon endpoint monitoring](#experiment-2)
- [Experiment 3: theHarvester email OSINT](#experiment-3)
- [Experiment 4: Netdiscover local host discovery](#experiment-4)
- [Experiment 5: Passive footprinting](#experiment-5)
- [Experiment 6: Google dorking](#experiment-6)
- [Experiment 7: FOCA document metadata](#experiment-7)
- [Experiment 8: Passive OSINT](#experiment-8)
- [Experiment 9: DNS infrastructure mapping](#experiment-9)
- [Experiment 10: WHOIS registration lookup](#experiment-10)
- [Experiment 11: nsLookup DNS analysis](#experiment-11)
- [Experiment 12: Sublist3r subdomain enumeration](#experiment-12)
- [Experiment 13: Maltego relationship graph](#experiment-13)
- [Experiment 14: Shodan Internet asset discovery](#experiment-14)
- [Experiment 15: Nikto web assessment](#experiment-15)
- [Experiment 16: Nmap network scanning](#experiment-16)
- [Experiment 17: Enum4linux SMB enumeration](#experiment-17)
- [Experiment 18: NetBIOS enumeration](#experiment-18)
- [Experiment 19: SNMPwalk enumeration](#experiment-19)
- [Experiment 20: SMTP user enumeration](#experiment-20)
- [Experiment 21: LDAP search](#experiment-21)
- [Experiment 22: DNSenum](#experiment-22)
- [Experiment 23: Nessus vulnerability assessment](#experiment-23)
- [Experiment 24: OpenVAS/GVM vulnerability assessment](#experiment-24)
- [Experiment 25: Wireshark packet analysis](#experiment-25)
- [Experiment 26: Burp Suite proxy analysis](#experiment-26)
- [Experiment 27: SQL injection and XSS lab testing](#experiment-27)
- [Experiment 28: Armitage exploitation management](#experiment-28)
- [Experiment 29: Robtex network intelligence](#experiment-29)
- [Experiment 30: pfSense firewall rules](#experiment-30)
- [Experiment 31: Snort NIDS](#experiment-31)

Placeholder decoder — how to find every `<LIKE_THIS>` value

Do this before running a command. Anything inside angle brackets is a note to replace, not text to type literally. First identify whether the command runs in the Kali VM, Windows host, browser, or a lab appliance. Use only the instructor-assigned lab and credentials.

1) Network placeholders (Kali VM)

Run these in Kali:

```
ip -br addr
ip route
ip -br link
```

Example:

```
eth0    UP    192.168.56.101/24
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.101
```

From this example only:

- <LAB_INTERFACE> = eth0 (the interface on the lab network)
- <LAB_SUBNET_CIDR> = 192.168.56.0/24 (the connected lab subnet)
- Kali's IP = 192.168.56.101

Do not copy the example values unless your output matches. If there are multiple interfaces, identify which one shares the lab network with the target. default via ... is the default route and is not automatically the correct interface for an isolated lab.

To ask Linux which route/interface it would use to reach a known target:

```
ip route get <TARGET_IP>
```

Find <TARGET_IP> from the target VM's own console:

```
ip -br addr
```

or, on an older Metasploitable target:

```
ifconfig
```

Look for its IPv4 inet address on the lab adapter. Then verify from Kali:

```
ping -c 4 <TARGET_IP>
ip neigh
```

If the target has no IP, check that both VMs are powered on and connected to the same Host-only/Internal Network in VirtualBox/VMware. Do not bridge a vulnerable target to a public network.

2) Domain and URL placeholders

- <DOMAIN>: use the domain explicitly assigned by your instructor (for example, example.com is a documentation example, not your real target). If the exercise is about a lab web app by IP, do not invent a domain; use the supplied hostname or URL.
- <AUTHORIZED_URL>: the full URL of the approved lab web app, including http:// or https://, e.g. http://192.168.56.102/ **only if that is the URL your lab actually uses**.
- <LAB_WEB_SERVER>: the approved lab web server's hostname/IP. For a local training app, use the URL shown in the lab instructions.
- <AUTHORIZED_NAMESERVER>: a name server assigned for the permitted DNS exercise. Find assigned name servers using dig <DOMAIN> NS; only test a server you are authorized to assess.

Commands that help check an assigned domain:

```
dig <DOMAIN> A
dig <DOMAIN> NS
dig <DOMAIN> MX
whois <DOMAIN>
```

For Windows PowerShell:

```
Resolve-DnsName <DOMAIN>
nslookup -type=NS <DOMAIN>
```

These commands query DNS/registration data; they do **not** tell you which domain the instructor intended. That value must come from your assignment.

3) Tool-specific placeholders

Placeholder • How to obtain it

<FILE_PATH> • The path to a capture/document supplied for the exercise. In Kali, use pwd, ls -lah, or find ~/ -type f \\(-iname '*.pcap' -o -iname '*.pcapng' \\) 2>/dev/null. In Windows PowerShell, use Get-ChildItem "$HOME\\Downloads" -Recurse -File. Use the full path if the file is outside the current folder.

<BASE_DN> • LDAP base distinguished name. First query the LDAP server Root DSE: ldapsearch -x -H ldap://<LDAP_SERVER> -s base -b "" namingContexts. Use a returned namingContexts value, or ask the directory administrator/instructor.

<LDAP_SERVER> • LDAP host/IP explicitly supplied by the instructor. Verify basic reachability with ip route get <LDAP_SERVER>; if permitted, nc -vz <LDAP_SERVER> 389 or nc -vz <LDAP_SERVER> 636. A port check does not discover the correct server.

<LAB_COMMUNITY> • SNMP community string supplied for the lab. It is a credential, not something to guess or brute-force. If none was supplied, ask the instructor.

<SNMP_USERNAME>, <AUTH_PASSPHRASE>, <PRIV_PASSPHRASE> • SNMPv3 credentials supplied by the lab administrator. Never substitute your Windows/Kali login unless the lab explicitly says to. Avoid saving real secrets in shell history or reports.

<SMTP_SERVER> / <LAB_SMTP_SERVER> • SMTP server hostname/IP in the lab handout. Check its name resolves with getent hosts <SMTP_SERVER> or Resolve-DnsName <SMTP_SERVER> on Windows. Only use the provided test-user list.

<LAB_SERVER_IP> • IP of the specific lab service/server named in that experiment. Find it on the server console (ip -br addr) or in the lab worksheet. Do not assume it is the same as Kali's IP.

<NETBIOS_NAME> • NetBIOS computer name provided by the lab or returned by an authorized NetBIOS lookup. On the Windows target, run hostname and, if enabled, nbtstat -n.

<USERNAME> • The test account specified by the instructor or returned by an authorized directory query. Do not guess real users or reuse personal account names.

<SNORT_CONF> • Snort configuration file. Check `snort -V`, `snort -h`, and package file locations. On Kali, try `dpkg -L snort 2>/dev/null • grep -E 'snort\\.conf$' and find /etc -name snort.conf 2>/dev/null`. Use the path that actually exists. Snort 2 and Snort 3 command syntax differs.

<RULE_FILE> • The rules file referenced by the actual Snort configuration. Inspect `include` lines in the config: `grep -n 'include' <SNORT_CONF>`. Use the real included rules path.

<LAB_INTERFACE> • The interface connected to the lab network; use `ip -br addr` and `ip route`. Common names include `eth0` or `ens33`, but names vary.

<LAB_SUBNET_CIDR> • Connected lab subnet shown by `ip route` (for example `192.168.56.0/24`). Confirm it includes the target and belongs to the isolated lab.

<TARGET_IP> • The target's address, obtained from its console or the instructor worksheet; validate with `ip route get <TARGET_IP>` and a permitted connectivity test.

<AUTHORIZED_URL> • Instructor-provided lab URL, checked in a browser before using Nikto/Burp.

<DOMAIN> • Instructor-assigned domain for OSINT/DNS tasks; do not replace it with a random public organization.

4) If the value cannot be discovered locally

Some placeholders are **credentials or scope decisions**, not technical facts that a command can safely infer. Ask your instructor/lab administrator for:

- the assigned domain and target IP;
- SNMP community string or SNMPv3 credentials;
- SMTP test server and approved test-user file;
- LDAP server and account permissions;
- the approved URL, scope, and scan permission.

5) Quick pre-flight checklist

```
# Kali: run these first
ip -br addr
ip route
ip neigh
```

```
# Windows host: run these first
Get-NetIPConfiguration
Get-NetRoute -AddressFamily IPv4
Get-NetNeighbor -AddressFamily IPv4
```

Before launching a scan, confirm: **(1)** target is assigned, **(2)** interface/subnet match the lab, **(3)** target is on the isolated lab network, and **(4)** the tool and command options match the installed version.

Experiment 1 — Configure a penetration testing environment by setting up an attacker machine and a vulnerable target machine. Verify connectivity and document network configuration.

Tool(s): Kali Linux, Metasploitable, VirtualBox/VMware

Run in: Kali VM terminal + VirtualBox/VMware GUI; Windows commands only when explicitly shown.

Keep vulnerable targets on a host-only/internal network. Do not bridge Metasploitable to a public or production network.

Kali networking checks

```
cat /etc/os-release
ip -br addr
ip route
ip neigh
ip link
hostname
```

Windows host checks

```
ipconfig /all
route print
arp -a
Get-NetIPConfiguration
Get-NetAdapter
```

Connectivity tests

```
ping -c 4 <TARGET_IP>
tracepath <TARGET_IP>
ip route get <TARGET_IP>
```

Lab-only host discovery

```
# Replace the example subnet with the isolated lab subnet.
sudo nmap -sn <LAB_SUBNET_CIDR>
sudo netdiscover -i <LAB_INTERFACE> -r <LAB_SUBNET_CIDR>
```

Evidence capture

```
ip -br addr > experiment1-addresses.txt
ip route > experiment1-routes.txt
ip neigh > experiment1-neighbors.txt
```

Quick interpretation

A failed ping can mean ICMP is blocked; verify routes, VM network mode, and target status before concluding the host is offline.

Experiment 2 — A Windows system is suspected of malware execution. Configure endpoint monitoring and collect process creation logs for forensic analysis.

Tool(s): Microsoft Sysmon (Windows)

Run in: Windows PowerShell (Run as administrator) and Windows Event Viewer.

Run installation and service commands from an elevated PowerShell session. Use a trusted Sysmon configuration supplied by your instructor or organization.

Download and version

```
Get-ComputerInfo | Select-Object WindowsProductName, WindowsVersion, OsArchitecture
# After downloading and extracting Sysmon from Microsoft Sysinternals:
.\Sysmon64.exe -?
```

Install or update Sysmon

```
# Install with the approved XML configuration file:
.\Sysmon64.exe -accepteula -i .\sysmonconfig.xml

# Update the active configuration:
.\Sysmon64.exe -c .\sysmonconfig.xml

# Display current configuration (supported by current Sysmon releases):
.\Sysmon64.exe -c
```

Service and event log checks

```
Get-Service Sysmon64,Sysmon -ErrorAction SilentlyContinue
Get-WinEvent -ListLog '*Sysmon*' | Select-Object LogName,RecordCount,IsEnabled
Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' -MaxEvents 20 |
  Select-Object TimeCreated,Id,LevelDisplayName,Message
```

Filter process creation and network events

```
Get-WinEvent -FilterHashtable @{
  LogName='Microsoft-Windows-Sysmon/Operational'; Id=1
} -MaxEvents 25 | Select-Object TimeCreated,Id,Message

# Event ID 3 appears only when network-connection logging is enabled:
Get-WinEvent -FilterHashtable @{
  LogName='Microsoft-Windows-Sysmon/Operational'; Id=3
} -MaxEvents 25 | Select-Object TimeCreated,Id,Message
```

Export evidence and uninstall

```
wevtutil epl Microsoft-Windows-Sysmon/Operational .\Sysmon-Operational.evtx
# Uninstall only when permitted by the lab administrator:
.\Sysmon64.exe -u
```

GUI path

Event Viewer → Applications and Services Logs → Microsoft → Windows → Sysmon → Operational. Event ID 1 is process creation; Event ID 3 is network connection and depends on configuration.

Experiment 3 — Identify publicly available employee email addresses using open-source intelligence.

Tool(s): theHarvester

Run in: Kali VM terminal.

theHarvester's supported data sources vary by release and may require API credentials. Use the help output from your installed version.

Install and verify

```
theHarvester -h
theHarvester --help
theHarvester -h | head -n 80
```

Domain search examples

```
# Substitute a domain assigned for the exercise and a source supported by your version:
theHarvester -d example.com -b duckduckgo
theHarvester -d example.com -b bing
```

Limit results and save output

```
# Options vary across releases; confirm -l, -b, and -f in local help.
theHarvester -d example.com -b bing -l 100
theHarvester -d example.com -b bing -f theharvester-example
```

Inspect saved results

```
ls -lh theharvester-example*
file theharvester-example*
```

Troubleshooting

If a source is unavailable, rate-limited, or requests an API key, choose a supported source shown by the installed version and document the limitation. Empty results do not prove that no public emails exist.

Experiment 4 — Discover active hosts on local network and identify IP/MAC addresses.

Tool(s): Netdiscover

Run in: Kali VM terminal.

ARP discovery is intended for the local Layer-2 segment. Run only on your assigned lab network.

Help and interface

```
netdiscover -h
ip -br addr
ip route
```

Passive and active discovery

```
# Passive mode (listening for ARP traffic):
sudo netdiscover -p -i <LAB_INTERFACE>

# Active ARP discovery of a specific lab subnet:
sudo netdiscover -i <LAB_INTERFACE> -r <LAB_SUBNET_CIDR>
```

Output and repeat checks

```
# Save terminal output when supported by shell redirection:
sudo netdiscover -i <LAB_INTERFACE> -r <LAB_SUBNET_CIDR> 2>&1 | tee netdiscover.txt
ip neigh
```

Troubleshooting

If no hosts appear, verify the interface and subnet, ensure both VMs share the same internal/host-only network, and confirm the target is powered on. ARP discovery will not normally discover hosts across routers.

Experiment 5 — Passive reconnaissance on assigned organization: domains, technologies, contact details, publicly available resources.

Tool(s): Footprinting techniques

Run in: Kali VM terminal for `whois`/`dig`; browser for passive public-source research.

For a strictly passive task, use public sources and avoid port scans or direct service probes unless the assignment explicitly permits them.

Registration and DNS lookups

```
whois <DOMAIN>
dig <DOMAIN> A
dig <DOMAIN> AAAA
dig <DOMAIN> NS
dig <DOMAIN> MX
dig <DOMAIN> TXT
```

Public HTTP metadata — active request

```
# This sends a request; do not use if the exercise requires strictly passive collection.
curl -I https://<DOMAIN>
```

Certificate and public-source workflow

Use the browser to inspect the organization's official website, published reports, public certificate-transparency records, and public DNS/registration data. Record source URL, collection time, and confidence.

Save command results

```
whois <DOMAIN> > whois.txt
dig <DOMAIN> ANY > dns-query.txt
```

Note

`ANY` responses are often incomplete or minimized; query specific record types instead when accuracy matters.

Experiment 6 — Advanced search operators reveal sensitive info exposed on Internet for given org; document risks.

Tool(s): Search engine operators

Run in: Browser/search engine; queries are pasted into the search box, not a terminal.

Search only the assigned organization. Do not attempt to access private records or exploit discovered content.

Domain and file searches

```
site:example.com
site:example.com filetype:pdf
site:example.com filetype:docx
site:example.com filetype:xlsx
site:example.com filetype:txt
```

Title and URL searches

```
site:example.com intitle:"index of"
site:example.com inurl:login
site:example.com intitle:login
site:example.com inurl:backup
```

Search for public documentation and exposed references

```
site:example.com "confidential"
site:example.com "internal use only"
site:example.com filetype:log
site:example.com filetype:sql
```

Evidence

Record the exact query, result URL, snippet, date, apparent exposure, and recommended action. Search operators and results differ across search engines and change over time.

Experiment 7 — Recover metadata from publicly available target organization documents and identify potential information leakage.

Tool(s): FOCA (GUI)

Run in: Windows FOCA GUI; Windows PowerShell only for the diagnostic checks.

FOCA is primarily a Windows GUI application; do not invent command-line switches for GUI features.

Workflow

Launch FOCA. 2. Create a new project. 3. Enter the authorized domain. 4. Choose available search engines/document types. 5. Search for public documents. 6. Collect permitted documents. 7. Run metadata analysis. 8. Review author, software, timestamps, usernames, paths, and other exposed fields. 9. Export or screenshot results.

Troubleshooting

If Google/Bing/DuckDuckGo searches fail, check connectivity, system date/time, the FOCA release, supported search-engine parsers, rate limits, and dependencies. A `429` means a provider is rate-limiting requests; it does not by itself indicate an incorrect domain.

Windows checks

```
Get-ComputerInfo | Select-Object WindowsProductName,WindowsVersion,OsArchitecture
Get-Date
Test-NetConnection www.google.com -Port 443
```

Evidence

Save the project/report and a screenshot of the metadata table. Only download documents that are public and within the exercise scope.

Experiment 8 — OSINT investigation of publicly available information without interacting with target systems.

Tool(s): Public OSINT sources and browser

Run in: Browser and Kali terminal for passive DNS/WHOIS lookups; avoid active probing.

This experiment explicitly requires passive collection. Do not use Nmap, Nikto, direct HTTP probing, or other active tests as a substitute.

Passive workflow

Search official organization pages, public reports, public business registries, published contact pages, public DNS/WHOIS data, and public certificate-transparency search interfaces.

DNS/registration queries

```
whois <DOMAIN>
dig <DOMAIN> A
dig <DOMAIN> NS
dig <DOMAIN> MX
```

Record evidence

```
Source URL:
Collection date/time:
Claim or observation:
Supporting evidence:
Confidence (high/medium/low):
Alternative explanation:
```

Validation

Cross-check important facts against a second independent public source. A public mention, shared IP, or old DNS record alone does not prove current ownership or control.

Experiment 9 — Identify DNS infrastructure: mail servers, authoritative name servers, associated records.

Tool(s): DNS mapping tools (`dig`, `host`, `nslookup`)

Run in: Kali VM terminal (`dig`/`host`); Windows `nslookup` where explicitly shown.

Query only records relevant to the assigned domain. DNS answers can differ by resolver and time.

Common record types

```
dig <DOMAIN> A
dig <DOMAIN> AAAA
dig <DOMAIN> MX
dig <DOMAIN> NS
dig <DOMAIN> SOA
dig <DOMAIN> TXT
dig <DOMAIN> CAA
```

Short output and alternate tools

```
dig +short <DOMAIN> A
dig +short <DOMAIN> MX
host -t NS <DOMAIN>
host -t MX <DOMAIN>
nslookup -type=MX <DOMAIN>
nslookup -type=NS <DOMAIN>
```

Query a chosen resolver

```
dig @1.1.1.1 <DOMAIN> A
dig @8.8.8.8 <DOMAIN> MX
```

Save and interpret

```
dig <DOMAIN> MX > mx-records.txt
dig <DOMAIN> NS > nameservers.txt
```

Interpretation

A/AAAA map to IPv4/IPv6 addresses; MX lists mail exchangers; NS lists name servers; SOA contains zone administration metadata; TXT can contain SPF or verification records.

Experiment 10 — Retrieve domain registration information, ownership details; analyze usefulness.

Tool(s): WHOIS

Run in: Kali VM terminal; browser alternative on Windows.

Registrant details may be privacy-redacted or omitted by registry policy.

Install and basic queries

```
whois --version
whois example.com
whois <DOMAIN>
```

Save output and search fields

```
whois <DOMAIN> | tee whois-output.txt
grep -Ei 'Registrar|Creation Date|Updated Date|Expiry|Name Server|Domain Status|Registrant' whois-output.txt
```

Connectivity and DNS comparison

```
dig <DOMAIN> NS
dig <DOMAIN> SOA
```

Windows alternative

Use a reputable WHOIS web service in the browser if no Windows WHOIS client is installed. Record the provider and lookup date.

Interpretation

Registrar, creation/expiry dates, status codes, and name servers may be visible. A redacted registrant field is unavailable information, not evidence of concealment or maliciousness.

Experiment 11 — Investigate DNS records; identify IPs, aliases, mail servers.

Tool(s): nsLookup / nslookup

Run in: Windows Command Prompt/PowerShell for `nslookup` examples; Kali terminal for `dig`.

Windows includes `nslookup`; Kali commonly provides it through DNS utilities. On Kali, `dig` is also useful.

Basic queries

```
nslookup example.com
nslookup <DOMAIN>
nslookup -type=A <DOMAIN>
nslookup -type=AAAA <DOMAIN>
nslookup -type=MX <DOMAIN>
nslookup -type=NS <DOMAIN>
nslookup -type=SOA <DOMAIN>
nslookup -type=TXT <DOMAIN>
```

Choose a DNS server

```
nslookup <DOMAIN> 1.1.1.1
nslookup <DOMAIN> 8.8.8.8
```

Interactive mode

```
nslookup
> set type=MX
> example.com
> set type=NS
> example.com
> exit
```

Kali equivalents

```
dig +short <DOMAIN> A
dig +short <DOMAIN> MX
dig +short <DOMAIN> CNAME
```

Troubleshooting

NXDOMAIN indicates the queried resolver says the name does not exist; SERVFAIL can indicate a resolution/authority issue. Check spelling and query a permitted alternate resolver.

Experiment 12 — Discover subdomains and prepare inventory.

Tool(s): Sublist3r

Run in: Kali VM terminal.

Sublist3r is an older project; source compatibility and dependencies can vary. Confirm options with the installed help.

Help and installation checks

```
sublist3r -h
python3 --version
command -v sublist3r
```

Basic and bounded enumeration

```
sublist3r -d example.com
sublist3r -d example.com -o subdomains.txt
sublist3r -d example.com -t 10
```

Inspect output

```
cat subdomains.txt
sort -u subdomains.txt -o subdomains.txt
wc -l subdomains.txt
```

Troubleshooting

If an option is rejected, check `sublist3r -h`; option availability depends on the installed version. Empty output may reflect source limitations or rate limits and does not prove no subdomains exist.

Experiment 13 — Create relationship graph among domains, IPs, DNS servers, organizations.

Tool(s): Maltego (GUI)

Run in: Maltego GUI on the machine where it is installed.

Maltego transform availability depends on the installed client, account, plan, credits, and data providers.

Graph workflow

Launch Maltego and sign in if required. 2. Create a new graph. 3. Add a Domain entity for the assigned domain. 4. Select the entity and inspect available transforms. 5. Run permitted transforms for DNS, IP, email, or organization relationships. 6. Expand only relevant results. 7. Open source/details for important links. 8. Save/export the graph.

Evidence

Capture the graph, the starting entity, selected transform, and source details for important relationships.

Interpretation

A graph link is a lead, not proof of common ownership. Verify shared IPs, domains, and email relationships independently.

Experiment 14 — Identify Internet-facing assets and assess publicly accessible services.

Tool(s): Shodan (browser)

Run in: Shodan website in a browser.

Use the assigned domain/IP scope. Shodan results are indexed observations and may be stale or incomplete.

Search examples

```
hostname:example.com
domain:example.com
ip:203.0.113.10
net:203.0.113.0/24
port:443 hostname:example.com
product:nginx hostname:example.com
```

Useful filters

```
hostname:example.com port:80
hostname:example.com port:443
ssl.cert.subject.cn:example.com
org:"Example Organization"
```

Evidence

Record the query, result IP, reported ports/services, banner/product, observation/last-seen context if shown, and date. Do not attempt to log into or exploit a discovered service.

Account limitations

Search depth, export, filters, and API access can depend on the account. Do not assume every query feature is available to free users.

Experiment 15 — Web server vulnerability assessment: outdated software, insecure config, potential web vulnerabilities.

Tool(s): Nikto

Run in: Kali VM terminal; authorized lab web server only.

Nikto sends HTTP requests and may be noisy. Scan only the assigned web server or lab application.

Install and help

```
nikto -Version
nikto -Help
```

Basic scans

```
nikto -h http://<AUTHORIZED_URL>
nikto -host <AUTHORIZED_URL>
nikto -h https://<AUTHORIZED_URL>
```

Port, tuning and output

```
# Verify option syntax with nikto -Help for your installed version.
nikto -h <AUTHORIZED_URL> -port 8080
nikto -h <AUTHORIZED_URL> -output nikto-report.txt
nikto -h <AUTHORIZED_URL> -Format html -output nikto-report.html
```

Review output

```
less nikto-report.txt
grep -Ei 'OSVDB|CVE|Server|header|error|vulnerable' nikto-report.txt
```

Interpretation

Treat version banners and reported paths as leads. Validate findings manually and recommend patching, removal of unnecessary files, secure headers, and reduced information disclosure.

Experiment 16 — Network scan live hosts, open ports, services, versions, OS.

Tool(s): Nmap

Run in: Kali VM terminal; authorized lab IP/subnet only.

Use only the instructor-assigned target or isolated lab subnet. More intensive options generate more traffic.

Version and help

```
nmap --version
nmap --help
man nmap
```

Host discovery

```
nmap -sn <LAB_SUBNET_CIDR>
nmap -sn <TARGET_IP>
nmap -Pn <TARGET_IP>
```

Port selection

```
nmap <TARGET_IP>
nmap -p 22,80,443 <TARGET_IP>
nmap -p 1-1024 <TARGET_IP>
nmap -p- <TARGET_IP>
nmap --top-ports 100 <TARGET_IP>
```

Service and OS detection

```
nmap -sV <TARGET_IP>
sudo nmap -O <TARGET_IP>
sudo nmap -sS <TARGET_IP>
nmap -sT <TARGET_IP>
sudo nmap -sU --top-ports 20 <TARGET_IP>
```

Timing and scripts

```
nmap -T3 -sV <TARGET_IP>
nmap -T2 -sV <TARGET_IP>
nmap -sC -sV <TARGET_IP>
```

Save results

```
nmap -oN nmap-report.txt <TARGET_IP>
nmap -oX nmap-report.xml <TARGET_IP>
nmap -oG nmap-report.gnmap <TARGET_IP>
nmap -oA nmap-report <TARGET_IP>
```

Read and troubleshoot

```
grep -E 'open|filtered|closed' nmap-report.txt
ip route get <TARGET_IP>
ping -c 4 <TARGET_IP>
```

Quick notes

`-sn` performs host discovery without a port scan; `-sV` probes services for version details; `-O` attempts OS detection and often needs elevated privileges; `-Pn` skips host discovery. Open/closed/filtered states are different and do not alone prove vulnerability.

Experiment 17 — Enumerate shares, users, groups, OS details from authorized Windows target.

Tool(s): enum4linux

Run in: Kali VM terminal; authorized Windows/SMB lab target only.

Run only against the authorized Windows/SMB lab host. Anonymous enumeration may be disabled.

Help and basic enumeration

```
enum4linux -h
enum4linux <TARGET_IP>
enum4linux -a <TARGET_IP>
```

Common enumeration categories

```
# Confirm supported flags with enum4linux -h; commonly supported in classic enum4linux:
enum4linux -U <TARGET_IP>
enum4linux -S <TARGET_IP>
enum4linux -G <TARGET_IP>
enum4linux -P <TARGET_IP>
enum4linux -o <TARGET_IP>
```

Save output

```
enum4linux -a <TARGET_IP> | tee enum4linux-report.txt
```

Troubleshooting

If SMB ports are unreachable, check the lab target and firewall. Access denied or empty results may indicate hardened SMB policy, not the absence of users or shares.

Experiment 18 — Identify NetBIOS names, shared resources, network info from assigned Windows system.

Tool(s): NetBIOS enumeration (`nbtscan`, `nbtstat`)

Run in: Kali terminal for `nbtscan`; Windows Command Prompt for `nbtstat`.

NetBIOS broadcasts and responses are usually limited to the local segment and may be disabled by modern Windows configuration.

Kali checks and scan

```
nbtscan -h
nbtscan <LAB_SUBNET_CIDR>
nbtscan <TARGET_IP>
```

Windows checks

```
nbtstat -n
nbtstat -c
nbtstat -A <TARGET_IP>
nbtstat -a <NETBIOS_NAME>
```

Save output

```
nbtscan <LAB_SUBNET_CIDR> | tee netbios-report.txt
```

Interpretation

`nbtstat -A` queries a remote IP; `-a` queries by NetBIOS name; `-n` lists local names; `-c` shows the name cache. A failed query may mean NetBIOS is disabled or filtered.

Experiment 19 — Retrieve management info from SNMP-enabled device and identify risks from exposed SNMP data.

Tool(s): SNMPwalk

Run in: Kali VM terminal; lab-provided SNMP credentials only.

Use only the community string or credentials provided for the authorized lab device. SNMPv1/v2c community strings are not encrypted.

Version and basic system tree

```
snmpwalk --version
snmpwalk -v2c -c <LAB_COMMUNITY> <TARGET_IP> 1.3.6.1.2.1.1
snmpwalk -v2c -c <LAB_COMMUNITY> <TARGET_IP> system
```

Specific common OIDs

```
snmpwalk -v2c -c <LAB_COMMUNITY> <TARGET_IP> 1.3.6.1.2.1.1.1.0
snmpwalk -v2c -c <LAB_COMMUNITY> <TARGET_IP> 1.3.6.1.2.1.1.3.0
snmpwalk -v2c -c <LAB_COMMUNITY> <TARGET_IP> 1.3.6.1.2.1.1.5.0
```

SNMPv3 pattern

```
# Replace credentials with lab-provided values; never place real secrets in shared notes.
snmpwalk -v3 -l authPriv -u <SNMP_USERNAME> -a SHA -A '<AUTH_PASSPHRASE>' -x AES -X '<PRIV_PASSPHRASE>'
<TARGET_IP> system
```

Save and interpret

```
snmpwalk -v2c -c <LAB_COMMUNITY> <TARGET_IP> system > snmp-system.txt
```

Troubleshooting

Timeout may mean SNMP is disabled, UDP/161 is filtered, or the credentials/version are wrong. Prefer SNMPv3 with authentication and privacy in production.

Experiment 20 — Assess SMTP server whether valid accounts can be enumerated; document findings and mitigations.

Tool(s): smtp-user-enum

Run in: Kali VM terminal; instructor-provided SMTP lab target and test list only.

Use only an explicitly authorized SMTP lab server and instructor-provided test usernames.

Help and modes

```
smtp-user-enum -h
```

VRFY mode

```
smtp-user-enum -M VRFY -U users.txt -t <SMTP_SERVER>
```

EXPN and RCPT modes

```
smtp-user-enum -M EXPN -U users.txt -t <SMTP_SERVER>
smtp-user-enum -M RCPT -U users.txt -t <SMTP_SERVER>
```

Alternate port and save output

```
# Confirm -p support in local help:
smtp-user-enum -M VRFY -U users.txt -t <SMTP_SERVER> -p 25
smtp-user-enum -M VRFY -U users.txt -t <SMTP_SERVER> | tee smtp-enum.txt
```

Interpretation

Server behavior can intentionally obscure account validity. Treat results as tentative unless the lab provides a known ground truth; mitigation includes disabling unnecessary VRFY/EXPN and normalizing responses.

Experiment 21 — Query LDAP directory to identify users, OUs, security info.

Tool(s): ldapsearch

Run in: Kali VM terminal; authorized LDAP directory only.

Use permitted lab access. For authenticated directories, prefer secure transport and avoid exposing passwords in shell history.

Version and connection

```
ldapsearch -VV
ldapsearch -x -H ldap://<LDAP_SERVER> -s base -b "" namingContexts
```

Search entries and users

```
ldapsearch -x -H ldap://<LDAP_SERVER> -b '<BASE_DN>' '(objectClass=*)' dn
ldapsearch -x -H ldap://<LDAP_SERVER> -b '<BASE_DN>' '(objectClass=person)' cn uid mail
ldapsearch -x -H ldap://<LDAP_SERVER> -b '<BASE_DN>' '(objectClass=organizationalUnit)' ou
```

Filters and selected attributes

```
ldapsearch -x -H ldap://<LDAP_SERVER> -b '<BASE_DN>' '(uid=<USERNAME>)' dn cn mail
ldapsearch -x -H ldap://<LDAP_SERVER> -b '<BASE_DN>' '(&(objectClass=person)(mail=*))' cn mail
```

Save output and secure connection

```
ldapsearch -x -H ldap://<LDAP_SERVER> -b '<BASE_DN>' '(objectClass=person)' dn cn mail > ldap-users.ldif
# Use ldaps:// or StartTLS only as configured by the directory administrator:
ldapsearch -x -H ldaps://<LDAP_SERVER> -b '<BASE_DN>' '(objectClass=person)' dn
```

Authentication

If bind authentication is required, consult local `ldapsearch` help and use the lab's approved credential-handling method; avoid placing plaintext passwords in shell history.

Experiment 22 — Comprehensive DNS enumeration: hostnames, subdomains, zone transfer vulnerabilities, DNS records.

Tool(s): DNSenum

Run in: Kali VM terminal; assigned DNS domain only.

Only test domains explicitly assigned for the exercise. Zone-transfer checks can reveal internal naming information.

Help and basic enumeration

```
dnsenum --help
dnsenum <DOMAIN>
```

Output and selected options

```
# Confirm exact options for the installed version before using:
dnsenum --enum <DOMAIN>
dnsenum --output dnsenum-report.xml <DOMAIN>
```

Independent record checks

```
dig <DOMAIN> A
dig <DOMAIN> MX
dig <DOMAIN> NS
dig <DOMAIN> SOA
```

Authorized zone-transfer test

```
# Run only against an instructor-authorized domain/name server:
dig AXFR <DOMAIN> @<AUTHORIZED_NAMESERVER>
```

Interpretation

A successful AXFR response can expose zone contents; a refused/failed transfer is expected for many correctly configured servers and does not prove the whole DNS setup is secure.

Experiment 23 — Vulnerability assessment, prioritize vulnerabilities by severity and recommend remediation.

Tool(s): Nessus (GUI)

Run in: Nessus GUI on the machine where Nessus is installed; PowerShell checks if installed on Windows.

Nessus requires installation, initialization, and the applicable activation/registration flow. Scan only assigned targets.

Windows service checks

```
Get-Service *nessus* -ErrorAction SilentlyContinue
Test-NetConnection localhost -Port 8834
```

GUI scan workflow

Open the local Nessus dashboard (commonly `https://localhost:8834`). 2. Complete activation and wait for initialization. 3. Create a scan using an appropriate template. 4. Enter only the authorized target. 5. Configure credentials only if approved. 6. Launch the scan. 7. Review severity, plugin output, affected asset and remediation. 8. Export the report.

Linux service checks, where Nessus is installed on Linux

```
sudo systemctl status nessusd
sudo systemctl start nessusd
sudo systemctl enable nessusd
```

Report and triage

Use the dashboard's export/report feature. Prioritize severity together with exposure, exploitability, asset importance, and compensating controls; validate important findings before describing them as confirmed.

Experiment 24 — Open-source vulnerability scan on provided network and generate professional report.

Tool(s): OpenVAS / Greenbone Vulnerability Management (GVM)

Run in: Kali VM terminal for GVM checks; Greenbone web GUI for scan setup/reporting.

GVM setup commands vary by Kali release. Follow the installed package's own diagnostic output and current official documentation.

Kali setup checks

```
command -v gvm-check-setup
gvm-check-setup
```

Service commands where supported

```
sudo gvm-start
sudo gvm-stop
```

Service status

```
systemctl --type=service | grep -Ei 'gvmd|gsad|ospd|openvas'
sudo systemctl status gvmd
sudo systemctl status gsad
```

GUI scan workflow

Open the local Greenbone web interface shown by the installed setup. Confirm feeds are synchronized, create a target containing only the assigned IP/subnet, create a task with a suitable scan configuration, start it, review results, and export the report.

Troubleshooting

Feed initialization can take time. If commands differ, follow `gvm-check-setup` and the package documentation for your release rather than copying service names from another distribution.

Experiment 25 — Analyze supplied packet capture, identify suspicious activity, protocols, evidence of malicious communication.

Tool(s): Wireshark / tshark

Run in: Wireshark GUI on the machine with the capture; Kali terminal for `tshark` commands.

Prefer the supplied capture file for offline analysis. Live capture can require elevated privileges or Npcap on Windows.

Version and interfaces

```
wireshark --version
tshark --version
tshark -D
```

Read capture and filter protocols

```
tshark -r <FILE_PATH>
tshark -r <FILE_PATH> -Y dns
tshark -r <FILE_PATH> -Y http
tshark -r <FILE_PATH> -Y 'tcp.port == 443'
tshark -r <FILE_PATH> -Y 'ip.addr == <TARGET_IP>'
```

Extract useful fields

```
tshark -r <FILE_PATH> -T fields -e frame.time -e ip.src -e ip.dst -e _ws.col.Protocol
tshark -r <FILE_PATH> -Y dns -T fields -e frame.time -e ip.src -e dns.qry.name
tshark -r <FILE_PATH> -Y 'tcp.flags.syn == 1 && tcp.flags.ack == 0'
```

Save filtered packets and live capture

```
tshark -r <FILE_PATH> -Y dns -w dns-only.pcapng
# Live capture on an authorized lab interface; stop with Ctrl+C:
sudo tshark -i <LAB_INTERFACE> -w lab-capture.pcapng
```

Wireshark display filters

```
dns
http
tcp
icmp
ip.addr == 192.0.2.10
tcp.port == 80
tcp.flags.syn == 1
dns.flags.rcode != 0
```

GUI actions

Select interface → Start capture → generate permitted test traffic → Stop → apply display filter → right-click packet → Follow → TCP Stream (where relevant) → File → Save As.

Experiment 26 — Intercept HTTP/HTTPS requests between client and vulnerable web app; identify insecure parameters and recommend controls.

Tool(s): Burp Suite

Run in: Burp Suite GUI on the machine where it is installed; use its built-in or dedicated test browser.

Use only an intentionally vulnerable local app or an explicitly authorized training target. Configure a dedicated browser profile.

Proxy setup

Open Burp → Temporary project or approved project → Proxy → Proxy settings. Confirm the listener address/port (commonly 127.0.0.1:8080). Use Burp's built-in browser or configure a dedicated test browser to use that proxy.

Request workflow

Proxy → Intercept → Intercept is on → browse to the authorized app → inspect request → Forward or Drop. Use Proxy → HTTP history to review captured requests. Right-click a request → Send to Repeater.

Repeater workflow

Repeater → edit one request parameter → Send → compare status code, response body, and application behavior. Keep test inputs non-destructive.

Local proxy checks

```
curl -I http://127.0.0.1:8080
```

HTTPS and cleanup

For HTTPS, trust Burp's CA only in the dedicated test browser/profile and follow PortSwigger's current instructions. Remove the temporary trust after the exercise. Do not intercept other users' traffic.

Experiment 27 — Assess provided web app for SQL Injection and XSS; demonstrate impact without modifying production data; recommend mitigations.

Tool(s): Burp Suite and manual testing

Run in: Burp Suite GUI and intentionally vulnerable local/authorized training application.

Perform testing only against a local intentionally vulnerable app or an explicitly authorized training target. Do not modify production data.

Baseline request

In Burp Proxy, capture a normal request and send it to Repeater. Save the baseline status code and response so changes can be compared.

Harmless SQLi indicators

```
'
''
' OR '1'='1
```

Use only as controlled input in a training lab. Observe errors, response changes, and server-side behavior; do not run destructive statements or extract unrelated data.

Harmless XSS proof in local lab

```
<script>alert(1)</script>
```

Use only in a deliberately vulnerable local test page. If script execution is blocked, record that behavior rather than bypassing protections on a real site.

Evidence and mitigation

Record the original request, modified test input, response/status, and screenshot. SQL injection: use parameterized queries and least-privilege database accounts. XSS: apply contextual output encoding, safe templating, input validation where appropriate, and a restrictive Content Security Policy as defense in depth.

Experiment 28 — Simulate pentest engagement by identifying vulnerable service and coordinating exploitation using GUI pentesting interface; document attack path and remediation.

Tool(s): Armitage / Metasploit

Run in: Armitage/Metasploit on Kali, if compatible; isolated vulnerable VM only.

Armitage is a legacy interface and may not work with current Kali/Metasploit versions. Use only an isolated intentionally vulnerable VM, with snapshots.

Compatibility and services

```
msfconsole --version
msfconsole -h
command -v armitage
```

Framework service checks (package/service names vary)

```
sudo systemctl status postgresql
sudo systemctl status metasploit
```

GUI workflow

Start the supported database/framework services for the installed release → launch Armitage using the documented method for that release → connect to the framework → import or discover only the lab target → review identified services → select a matching module → read module options and required settings → verify target and payload settings → run only the instructor-approved lab exercise → save evidence.

Post-exercise

Record the path and observed impact. Stop the lab activity and restore the target snapshot. Do not run exploit modules against public or unrelated systems.

Experiment 29 — Investigate network reputation and historical DNS of assigned domain to identify infrastructure relationships.

Tool(s): Robtex (browser)

Run in: Robtex website in a browser; Kali terminal for DNS/WHOIS cross-checks.

Robtex is a browser-based service; features and data availability may change.

Search workflow

Open the official Robtex site → search the assigned domain or IP → inspect DNS records, related IPs/domains, and historical DNS/reputation sections available to your account → record source and date.

Cross-check with DNS

```
dig <DOMAIN> A
dig <DOMAIN> NS
dig -x <TARGET_IP>
whois <TARGET_IP>
```

Interpretation

Shared hosting, cloud infrastructure, and historical DNS can create relationships that do not prove common ownership. Cross-check findings with authoritative sources.

Experiment 30 — Configure firewall for secure access with inbound/outbound rules; verify unauthorized traffic blocked while legitimate services remain accessible.

Tool(s): pfSense (GUI)

Run in: pfSense webConfigurator GUI; test client commands run on the specified Windows/Kali client.

Make changes in an isolated lab. Ensure console access or a recovery snapshot is available before editing rules.

GUI rule workflow

Status → Interfaces: confirm interface addresses. Firewall → Rules → choose the relevant interface → Add rule → choose action (Pass/Block/Reject), protocol, source, destination, destination port, description → Save → Apply Changes.

NAT and logs

Firewall → NAT → Port Forward only when required by the lab. Status → System Logs → Firewall to inspect blocked/allowed traffic, where logging is enabled.

Verification from client

```
ping <TARGET_IP>
nc -vz <TARGET_IP> 80
nc -vz <TARGET_IP> 443
```

Windows verification

```
Test-NetConnection <TARGET_IP> -Port 80
Test-NetConnection <TARGET_IP> -Port 443
```

Rules and safety

Rules are evaluated in order on the relevant interface. Test both an allowed service and a denied service. Avoid locking out the management interface; change one rule at a time.

Experiment 31 — Configure NIDS to detect suspicious activity; generate alerts for simulated attacks and analyze logged events.

Tool(s): Snort

Run in: Kali VM terminal; check Snort major version before using Snort 2-style commands.

Snort command syntax differs between major versions and packages. Validate configuration before starting a sensor.

Version and help

```
snort -V
snort -h
```

Configuration test

```
# Snort 2-style syntax; check local help/config paths first:
sudo snort -T -c <SNORT_CONF> -i <LAB_INTERFACE>
```

Read a PCAP offline

```
# Use only the syntax supported by the installed version:
sudo snort -r <FILE_PATH> -c <SNORT_CONF>
```

Live lab monitoring

```
# Snort 2-style example; run only on an authorized lab interface:
sudo snort -i <LAB_INTERFACE> -c <SNORT_CONF> -A console
```

Rule checks

```
grep -n 'alert ' <RULE_FILE>
grep -n 'include' <SNORT_CONF>
```

Evidence and troubleshooting

Save alert output and relevant timestamps. Check interface selection, configuration path, included rules, permissions, and version-specific command syntax. Use benign instructor-provided test traffic or a supplied PCAP; do not generate attacks against unrelated networks.

Cross-experiment quick commands

Kali system and network checks

```
cat /etc/os-release
ip -br addr
ip route
ip neigh
ss -lntup
command -v nmap
man nmap
```

Windows checks

```
Get-ComputerInfo | Select-Object WindowsProductName,WindowsVersion,OsArchitecture
Get-NetIPConfiguration
Get-Service
Get-WinEvent -ListLog '*Sysmon*'
Test-NetConnection example.com -Port 443
```

Safe output capture

```
mkdir -p evidence
some-command 2>&1 | tee evidence/experiment-output.txt
```

Replace `some-command` with the actual command for the experiment. Redirection captures output; it does not validate findings.

Last-minute reminders

- `A` = IPv4 DNS record; `AAAA` = IPv6; `MX` = mail exchanger; `NS` = name server; `CNAME` = alias; `TXT` = text/policy data.
- Nmap: `-sn` host discovery; `-p` port selection; `-sV` service/version detection; `-O` OS detection; `-oN` normal output; `-oA` multiple output formats.
- Wireshark display filters: `dns`, `http`, `tcp`, `icmp`, `ip.addr == ...`, `tcp.port == ...`.
- Sysmon Event ID 1 = process creation; Event ID 3 = network connection when configured.
- An empty result does not prove absence. Check input, scope, permissions, service status, filters, version, and external-source limits.
- Vulnerability scanner findings are leads until validated. Record severity, evidence, limitations, and remediation.
- Restore snapshots and undo lab-only configuration changes after intrusive exercises.

Expanded practical-exam runbooks (additional material)

These sections extend the quick command lists above with more step-by-step Windows workflows, common examiner variations, result checks, and evidence to save. They supplement the matching numbered experiment; they do not replace it.

Experiment 1 — Extra lab setup and connectivity workflow

Run in: Kali VM terminal + VirtualBox/VMware GUI; Windows commands only when explicitly shown.

VirtualBox GUI steps

Open VirtualBox → select the Kali VM → **Settings** → **Network**.

For an isolated two-VM lab, choose **Host-only Adapter** or **Internal Network** on the adapter used by both VMs. Use the same network name for both VMs.

Start Kali and Metasploitable. In Kali, run:

```
ip -br addr
ip route
```

Find the target address from the target console using `ifconfig` or `ip addr` if available.

Verify the subnet matches, then run:

```
ping -c 4 <TARGET_IP>
ip neigh
```

If permitted, confirm the target's services with a limited lab scan:

```
nmap -sV -p 21,22,23,25,53,80,139,445 <TARGET_IP>
```

Save `ip -br addr`, `ip route`, ping output, and the limited scan to the experiment evidence folder.

Windows host network checks

```
ipconfig /all
Get-NetIPConfiguration
Get-NetRoute -AddressFamily IPv4
Get-NetNeighbor -AddressFamily IPv4
route print
arp -a
Test-NetConnection <TARGET_IP>
Test-NetConnection <TARGET_IP> -Port 80
```

Common failures

- **No IP:** check VM adapter mode and DHCP configuration.
- **Different subnets:** place both VMs on the same intended lab network or configure routing deliberately.
- **Ping fails:** test a TCP service too; ICMP may be blocked.
- **No route:** inspect `ip route` / `route print` and the VM adapter settings.

Experiment 2 — Sysmon: detailed Windows practical

Run in: Windows PowerShell (Run as administrator) and Windows Event Viewer.

Open an elevated PowerShell window

Start menu → type **PowerShell** → right-click **Windows PowerShell** → **Run as administrator**. Change directory to the folder containing the extracted Sysmon files:

```
Set-Location "$HOME\Downloads\Sysmon"
Get-ChildItem
.\Sysmon64.exe -?
```

Install, update, check, and remove

```
# Install with the lab-approved configuration:
.\Sysmon64.exe -accepteula -i .\sysmonconfig.xml

# Show configuration / update it:
.\Sysmon64.exe -c
.\Sysmon64.exe -c .\sysmonconfig.xml

# Check service and event channel:
Get-Service Sysmon64,Sysmon -ErrorAction SilentlyContinue
Get-WinEvent -ListLog 'Microsoft-Windows-Sysmon/Operational'

# Export the event channel:
wevtutil epl Microsoft-Windows-Sysmon/Operational "$PWD\Sysmon-Operational.evtx"

# Uninstall only if permitted:
.\Sysmon64.exe -u
```

Event Viewer: exact GUI path

Press **Win+R**, type `eventvwr.msc`, press Enter.

Open **Applications and Services Logs** → **Microsoft** → **Windows** → **Sysmon** → **Operational**.

Select **Filter Current Log…** in the right pane.

Enter event IDs such as `1` for Process Create or `3` for Network Connect, if network events are enabled in the active configuration.

Select an event and inspect **General** and **Details** → **XML View**.

Use **Save Filter to Custom View** if you need to reuse the filter.

PowerShell event queries

```
# Recent process-creation events (ID 1):
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=1} -MaxEvents 50 |
  Format-List TimeCreated,Id,Message

# Recent network-connection events (ID 3; only if configured):
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=3} -MaxEvents 50 |
  Format-List TimeCreated,Id,Message

# Search events mentioning a process name:
Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' -MaxEvents 500 |
  Where-Object {$_.Message -match 'powershell.exe'} |
  Select-Object TimeCreated,Id,Message

# Export a readable CSV:
Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' -MaxEvents 500 |
  Select-Object TimeCreated,Id,ProviderName,Message |
  Export-Csv .\sysmon-events.csv -NoTypeInformation
```

Exam evidence: Sysmon version, active configuration, selected event IDs, one or more relevant event records, and exported `.evtx`/CSV.

Experiment 3 — theHarvester: practical variations

Run in: Kali VM terminal.

```
theHarvester -h
theHarvester -d example.com -b all
theHarvester -d example.com -b bing
theHarvester -d example.com -b duckduckgo
theHarvester -d example.com -b crtsh
theHarvester -d example.com -l 200
theHarvester -d example.com -f example-results
```

The source names and option support vary by release. If `all` or a source is rejected, use the installed help to see supported sources. Do not interpret source failures or zero results as proof that no public records exist.

Experiment 4 — Netdiscover: practical variations

Run in: Kali VM terminal.

```
ip -br link
ip -br addr
ip route
sudo netdiscover -p
sudo netdiscover -i eth0
sudo netdiscover -i eth0 -r 192.168.56.0/24
sudo netdiscover -i eth0 -r 192.168.56.0/24 -p
sudo netdiscover -i eth0 -r 192.168.56.0/24 2>&1 | tee netdiscover.txt
ip neigh show
```

Replace `eth0` with the interface shown by `ip -br link`. Use only the assigned subnet.

Experiment 5 — Footprinting: extra passive commands

Run in: Kali VM terminal for `whois`/`dig`; browser for passive public-source research.

```
whois example.com
dig example.com A +short
dig example.com AAAA +short
dig example.com NS +short
dig example.com MX +short
dig example.com TXT +short
dig example.com CAA +short
host -a example.com
```

`host -a` can return incomplete results and should not replace specific record queries. For a passive-only assignment, do not add active scans simply because they appear elsewhere in this handbook.

Experiment 6 — Search operators: exam-ready query set

Run in: Browser/search engine; queries are pasted into the search box, not a terminal.

Paste one query at a time into a search engine, changing `example.com` to the assigned domain:

```
site:example.com
site:example.com filetype:pdf
site:example.com filetype:docx
site:example.com filetype:xlsx
site:example.com filetype:pptx
site:example.com filetype:txt
site:example.com intitle:"index of"
site:example.com inurl:login
site:example.com inurl:admin
site:example.com inurl:backup
```

```
site:example.com "internal use only"
site:example.com "confidential"
site:example.com "password"
```

Do not try credentials or download/access private data. Capture the query, visible result URL, date, and risk explanation. Search results may be stale or false positives.

Experiment 7 — FOCA on Windows: click-by-click workflow

Run in: Windows FOCA GUI; Windows PowerShell only for the diagnostic checks.

Before opening FOCA

Extract the FOCA release into a normal folder, such as `C:\Tools\FOCA`.

Read the release's included requirements/README. Do not assume the newest SQL Server release is compatible with an older FOCA build.

If Windows blocks the executable, verify its origin and digital signature before allowing it. Do not disable system protections blindly.

Open FOCA from the extracted folder or Start menu if an installer created a shortcut.

Project and metadata workflow

Open FOCA → **Project** → **New Project** (menu names may differ slightly by version).

Enter a project name and the authorized organization domain.

Choose the available search engines and file types in the project/search settings.

Start the document search and inspect discovered URLs.

Select permitted documents and use the available download/metadata-analysis action.

Review metadata fields such as author, creator, application, timestamps, usernames, local paths, and document properties.

Save the project and export the report if the installed version provides that option.

Capture screenshots of the domain, search results, document list, and metadata findings.

Windows diagnostics if search fails

```
Get-Date
Test-NetConnection www.google.com -Port 443
Test-NetConnection www.bing.com -Port 443
Test-NetConnection duckduckgo.com -Port 443
Resolve-DnsName www.google.com
```

A `429 Too Many Requests` response usually indicates rate limiting. A provider/parser error can be caused by FOCA's age, changed search-engine pages, network restrictions, or dependencies. Try another available provider, wait before retrying, and record the limitation; do not claim the tool is working if no results are returned.

Experiment 8 — Passive OSINT: collection checklist

Run in: Browser and Kali terminal for passive DNS/WHOIS lookups; avoid active probing.

For each source, record:

- source URL and access date/time;
- exact fact collected;
- screenshot or exported evidence where allowed;
- whether the source is primary or secondary;
- confidence and a second source for cross-checking.

Use public official pages, public registry data, published documents, public DNS/WHOIS, and public certificate-transparency sources. Do not perform direct service probing in a passive-only exercise.

Experiment 9 — DNS infrastructure: additional queries

Run in: Kali VM terminal (`dig`/`host`); Windows `nslookup` where explicitly shown.

```
for type in A AAAA MX NS SOA TXT CAA; do
  echo "===== $type ====="
  dig +noall +answer example.com "$type"
done

dig +trace example.com
dig @1.1.1.1 example.com A +short
dig @8.8.8.8 example.com MX +short
host -t MX example.com
host -t NS example.com
```

`dig +trace` makes DNS queries through the delegation path. Compare answers carefully; different resolvers and TTLs can explain differences.

Experiment 10 — WHOIS: more checks

Run in: Kali VM terminal; browser alternative on Windows.

```
whois example.com
whois 203.0.113.10
whois example.com | tee whois.txt
grep -Ei 'registrar|creation|updated|expiry|name server|status|country|org' whois.txt
```

IP WHOIS reports the network allocation holder, not necessarily the specific device operator. Registration records may be redacted.

Experiment 11 — nslookup: Windows interactive practice

Run in: Windows Command Prompt/PowerShell for `nslookup` examples; Kali terminal for `dig`.

Open **Command Prompt** or **PowerShell** from Start and run:

```
nslookup example.com
nslookup -type=A example.com
nslookup -type=AAAA example.com
nslookup -type=MX example.com
nslookup -type=NS example.com
nslookup -type=SOA example.com
nslookup -type=TXT example.com
nslookup example.com 1.1.1.1
```

Interactive mode:

```
nslookup
set type=MX
example.com
set type=NS
example.com
server 1.1.1.1
example.com
exit
```

Windows equivalent checks:

```
Resolve-DnsName example.com
Resolve-DnsName example.com -Type MX
Resolve-DnsName example.com -Type NS
Clear-DnsClientCache
ipconfig /displaydns
```

`Clear-DnsClientCache` changes local cache state; use only when useful for the lab.

Experiment 12 — Sublist3r: output handling and validation

Run in: Kali VM terminal.

```
sublist3r -h
sublist3r -d example.com -o subdomains.txt
sort -u subdomains.txt -o subdomains.txt
wc -l subdomains.txt
cat subdomains.txt
```

Validate selected names using passive DNS lookups:

```
while read -r name; do
  printf '%s ' "$name"
  dig +short "$name" A | paste -sd ',' -
done < subdomains.txt
```

This sends DNS queries. Use only if DNS validation is permitted by the assignment.

Experiment 13 — Maltego: detailed GUI path

Run in: Maltego GUI on the machine where it is installed.

Open Maltego → sign in or select the available community/trial option.

Choose **New Graph**.

From the entity palette, drag a **Domain** entity onto the graph and set it to the assigned domain.

Right-click the entity → inspect the available **Run Transform(s)** options.

Run only transforms available to your account; some require configuration, credits, or third-party access.

Expand results and select relevant entities (DNS name, IP address, email address, organization).

Right-click selected entities to run further transforms only when relevant.

Open entity details/source information to validate the relationship.

Use the graph layout controls to make the relationships readable.

Save the graph and export an image/report if the installed edition supports it.

In the report, distinguish confirmed facts from inferred relationships.

Experiment 14 — Shodan: practical query set

Run in: Shodan website in a browser.

```
hostname:example.com
domain:example.com
hostname:example.com port:80
hostname:example.com port:443
hostname:example.com port:22
hostname:example.com product:nginx
hostname:example.com product:Apache
ssl.cert.subject.cn:example.com
net:203.0.113.0/24
```

In the Shodan website, search the query bar → open a result → record the IP, port, service/banner, and observation date shown. Search filters and result detail depend on account access. Do not attempt to connect to or exploit services found in search results.

Experiment 15 — Nikto: extra command patterns

Run in: Kali VM terminal; authorized lab web server only.

```
nikto -Help
nikto -h http://<LAB_WEB_SERVER>
nikto -h https://<LAB_WEB_SERVER>
nikto -h <LAB_WEB_SERVER> -p 80
nikto -h <LAB_WEB_SERVER> -p 443
nikto -h <LAB_WEB_SERVER> -output nikto.txt
nikto -h <LAB_WEB_SERVER> -Format html -output nikto.html
nikto -h <LAB_WEB_SERVER> -timeout 10
nikto -h <LAB_WEB_SERVER> -Display 1234
```

Check `nikto -Help` for supported `-Display` values and format names in your installed version. Scanning is noisy; keep it within the assigned lab.

Experiment 16 — Nmap: common examiner variations

Run in: Kali VM terminal; authorized lab IP/subnet only.

```
# Discovery and basic scan
nmap -sn <LAB_SUBNET_CIDR>
nmap -Pn <TARGET_IP>
nmap <TARGET_IP>

# TCP scan styles
sudo nmap -sS <TARGET_IP>
nmap -sT <TARGET_IP>
nmap -p 22,80,443 <TARGET_IP>
nmap -p 1-1000 <TARGET_IP>
nmap -p- <TARGET_IP>
nmap --top-ports 100 <TARGET_IP>

# Service, OS, scripts, and timing
nmap -sV <TARGET_IP>
sudo nmap -O <TARGET_IP>
nmap -sC -sV <TARGET_IP>
nmap -T2 <TARGET_IP>
nmap -T3 -sV <TARGET_IP>

# UDP sample; UDP scans can be slow
sudo nmap -sU --top-ports 20 <TARGET_IP>

# Output formats
nmap -oN result.txt <TARGET_IP>
nmap -oX result.xml <TARGET_IP>
nmap -oG result.gnmap <TARGET_IP>
nmap -oA result <TARGET_IP>

# Confirm a specific port/service
nmap -sV -p 22,80,443 <TARGET_IP>
```

Do not use intrusive NSE scripts, evasion options, or broad public scans for a routine practical. Confirm the target and scope before starting.

Experiment 17 — enum4linux: output and options

Run in: Kali VM terminal; authorized Windows/SMB lab target only.

```
enum4linux -h
enum4linux -a <TARGET_IP>
enum4linux -U <TARGET_IP>
```

```
enum4linux -S <TARGET_IP>
enum4linux -G <TARGET_IP>
enum4linux -P <TARGET_IP>
enum4linux -o <TARGET_IP>
enum4linux -a <TARGET_IP> 2>&1 | tee enum4linux.txt
```

The classic `enum4linux` and newer `enum4linux-ng` have different options. If the installed tool is `enum4linux-ng`, run `enum4linux-ng -h` and use its documented syntax.

Experiment 18 — NetBIOS: Windows GUI and commands

Run in: Kali terminal for `nbtscan`; Windows Command Prompt for `nbtstat`.

Command Prompt

Start → type **cmd** → Enter:

```
nbtstat -n
nbtstat -c
nbtstat -R
nbtstat -A <TARGET_IP>
nbtstat -a <NETBIOS_NAME>
ipconfig /all
arp -a
```

`nbtstat -R` purges and reloads the NetBIOS name cache; use only if cache reset is part of the exercise.

Kali

```
nbtscan -h
nbtscan <TARGET_IP>
nbtscan <LAB_SUBNET_CIDR>
nbtscan -r <LAB_SUBNET_CIDR>
```

Check `nbtscan -h` because switches can differ by package. Use only the lab subnet.

Experiment 19 — SNMPwalk: practical OID variations

Run in: Kali VM terminal; lab-provided SNMP credentials only.

```
snmpwalk -h
snmpwalk -v2c -c <LAB_COMMUNITY> <TARGET_IP> system
snmpwalk -v2c -c <LAB_COMMUNITY> <TARGET_IP> 1.3.6.1.2.1.1
snmpget -v2c -c <LAB_COMMUNITY> <TARGET_IP> 1.3.6.1.2.1.1.1.0
snmpget -v2c -c <LAB_COMMUNITY> <TARGET_IP> 1.3.6.1.2.1.1.5.0
snmpwalk -v2c -c <LAB_COMMUNITY> <TARGET_IP> 1.3.6.1.2.1.2
snmpwalk -v2c -c <LAB_COMMUNITY> <TARGET_IP> system > snmp.txt
```

Use lab-provided credentials only. A timeout can be caused by network filtering, a disabled service, the wrong SNMP version, or the wrong community string.

Experiment 20 — SMTP user enumeration: checks and evidence

Run in: Kali VM terminal; instructor-provided SMTP lab target and test list only.

```
smtp-user-enum -h
smtp-user-enum -M VRFY -U users.txt -t <LAB_SMTP_SERVER>
smtp-user-enum -M EXPN -U users.txt -t <LAB_SMTP_SERVER>
smtp-user-enum -M RCPT -U users.txt -t <LAB_SMTP_SERVER>
smtp-user-enum -M VRFY -U users.txt -t <LAB_SMTP_SERVER> -p 25
smtp-user-enum -M VRFY -U users.txt -t <LAB_SMTP_SERVER> 2>&1 | tee smtp-user-enum.txt
```

Check `-h` for available options and port syntax. Use a small instructor-provided test list, not a real employee list. Document whether responses distinguish valid and invalid lab accounts.

Experiment 21 — LDAP search: practical filter variations

Run in: Kali VM terminal; authorized LDAP directory only.

```
ldapsearch -x -H ldap://<LDAP_SERVER> -s base -b "" namingContexts
ldapsearch -x -H ldap://<LDAP_SERVER> -b '<BASE_DN>' '(objectClass=*)' dn
ldapsearch -x -H ldap://<LDAP_SERVER> -b '<BASE_DN>' '(objectClass=person)' cn sn uid mail
ldapsearch -x -H ldap://<LDAP_SERVER> -b '<BASE_DN>' '(objectClass=organizationalUnit)' ou
ldapsearch -x -H ldap://<LDAP_SERVER> -b '<BASE_DN>' '(uid=<USERNAME>)' dn cn mail
ldapsearch -x -H ldap://<LDAP_SERVER> -b '<BASE_DN>' '(&(objectClass=person)(mail=*))' cn mail
ldapsearch -x -H ldaps://<LDAP_SERVER> -b '<BASE_DN>' '(objectClass=person)' dn cn mail
```

For a bind-required lab, use instructor-approved authentication and secure transport. Do not place real passwords directly in shell history.

Experiment 22 — DNSenum: extra record and zone checks

Run in: Kali VM terminal; assigned DNS domain only.

```
dnsenum --help
```

```
dnsenum <DOMAIN>
dnsenum --enum <DOMAIN>
dnsenum --output dnsenum.xml <DOMAIN>
dig <DOMAIN> A
dig <DOMAIN> MX
dig <DOMAIN> NS
dig <DOMAIN> SOA
dig AXFR <DOMAIN> @<AUTHORIZED_NAMESERVER>
```

Only run AXFR against an explicitly assigned name server. A denied transfer is normal for many domains.

Experiment 23 — Nessus on Windows: installation and scan steps

Run in: Nessus GUI on the machine where Nessus is installed; PowerShell checks if installed on Windows.

Start and open Nessus

Install Nessus using the installer from the official Tenable site and follow the installer prompts.

Open **Services** (`services.msc`) and check the Tenable Nessus service if the UI does not load.

Open a browser and navigate to `https://localhost:8834` on the same computer, unless the installation specifies a different URL.

Complete first-run setup, activation, and plugin initialization. Wait until initialization finishes.

Sign in with the local Nessus account created during setup.

Create and run a scan

Select **New Scan**.

Choose the scan template required by the practical (for example, a basic network scan).

Enter a name and the assigned target IP/subnet only.

Configure credentials only if the instructor has provided them and the task calls for authenticated scanning.

Save → **Launch**.

Open the completed scan → inspect **Vulnerabilities** and **Hosts**.

Open each important finding and record severity, plugin evidence, affected port/service, and remediation.

Use **Export** to save the report in the required format, if available in the edition.

Windows troubleshooting

```
Get-Service *nessus* -ErrorAction SilentlyContinue
Test-NetConnection localhost -Port 8834
Get-NetTCPConnection -LocalPort 8834 -State Listen -ErrorAction SilentlyContinue
```

If the dashboard is unavailable, check the service state, initialization progress, browser certificate warning for the local service, and firewall policy. Do not disable protections globally.

Experiment 24 — OpenVAS/GVM: detailed setup and scan workflow

Run in: Kali VM terminal for GVM checks; Greenbone web GUI for scan setup/reporting.

Kali commands

```
gvm-check-setup
sudo gvm-start
sudo gvm-stop
systemctl --type=service | grep -Ei 'gvmd|gsad|ospd|openvas'
sudo systemctl status gvmd
sudo systemctl status gsad
```

Use the setup script's instructions for the installed Kali version; service names and web addresses can change.

GUI workflow

Run the setup/check command and resolve reported missing components.

Start GVM using the supported command for the installed release.

Open the web address printed by the startup output.

Wait for feeds to finish synchronizing.

Go to **Configuration** → **Targets** (wording may vary) and create a target containing only the assigned lab host/subnet.

Go to **Scans** → **Tasks** and create a task for that target.

Select the instructor-required scan configuration; save and start.

Open the finished task → Results → inspect severity, NVT/details, affected host/port, and solution.

Export the report using the available report format.

Record scan date, target scope, feed status, scan configuration, and any limitations.

Experiment 25 — Wireshark: Windows install, capture, and analysis

Run in: Wireshark GUI on the machine with the capture; Kali terminal for `tshark` commands.

Install and open

Download Wireshark for Windows from the official Wireshark site.

Run the installer. Accept the Npcap installation prompt if live packet capture is needed and permitted.

Finish installation and launch **Wireshark** from the Start menu.

To inspect the supplied capture: **File** → **Open** → choose the `.pcap` or `.pcapng` file.

To capture live traffic: choose the correct Ethernet/Wi-Fi/lab interface from the start page → click the shark-fin Start button → reproduce only permitted lab traffic → click Stop.

Display filters to try

```
dns
http
tls
tcp
udp
icmp
arp
ip.addr == 192.0.2.10
ip.src == 192.0.2.10
ip.dst == 192.0.2.10
tcp.port == 80
tcp.port == 443
dns.flags.rcode != 0
tcp.flags.syn == 1 && tcp.flags.ack == 0
http.request
```

Useful GUI actions

- **Statistics** → **Protocol Hierarchy**: protocol breakdown.
- **Statistics** → **Conversations**: talkers and traffic volume.
- **Statistics** → **Endpoints**: hosts observed.
- **Statistics** → **I/O Graphs**: traffic over time.
- Select a packet → expand protocol fields in the middle pane.
- Right-click a relevant packet → **Follow** → **TCP Stream** where applicable.
- **File** → **Export Specified Packets** to save a filtered subset.
- **File** → **Save As** to preserve a working copy of the capture.

Windows capture checks

```
Get-NetAdapter
ipconfig /all
```

If no interfaces are listed, check Npcap installation and permissions. A capture may contain sensitive data; store it only in the authorized evidence location.

Experiment 26 — Burp Suite on Windows: full GUI workflow

Run in: Burp Suite GUI on the machine where it is installed; use its built-in or dedicated test browser.

Install Burp Suite Community Edition from the official PortSwigger download page.

Launch Burp Suite from the Start menu.

Select the temporary project option if appropriate → **Use Burp defaults** → **Start Burp**.

Open **Proxy** → **Intercept** and confirm the listener is enabled (commonly `127.0.0.1:8080`).

Choose **Open browser** from Burp Proxy to use Burp's preconfigured browser, or configure a dedicated test browser's HTTP proxy to `127.0.0.1`, port `8080`.

Browse only to the authorized training app. Requests should appear under **Proxy** → **HTTP history**.

With Intercept on, inspect a request; click **Forward** to send or **Drop** to discard.

Right-click a request in HTTP history → **Send to Repeater**.

In **Repeater**, change one non-destructive parameter at a time → **Send** → compare status, headers, and body.

Use **Target** → **Site map** to review paths observed while browsing.

Save a project or export selected evidence if supported by your edition.

Common issues

- **Browser does not connect:** verify proxy listener and browser proxy settings.
- **HTTPS warning:** use Burp's browser or follow PortSwigger's current CA instructions in a dedicated profile only.
- **Page hangs:** turn Intercept off or forward the pending request.
- **No requests appear:** check the browser is using the Burp proxy and the right listener port.

Experiment 27 — Burp manual SQLi/XSS practical: safe sequence

Run in: Burp Suite GUI and intentionally vulnerable local/authorized training application.

Start the authorized vulnerable app and Burp.

Browse the app through Burp and capture a baseline request.

Send the request to **Repeater**.

Identify one input parameter and change only that parameter.

For a lab SQL injection check, use a harmless quote or boolean test supplied by the instructor; compare the response with baseline.

For a lab reflected-XSS check, use a benign proof-of-execution payload only on the intentionally vulnerable app.

Record the request, response, observed difference, and screenshot.

Do not dump databases, alter records, upload persistent scripts, or test production data.

Recommend parameterized SQL queries, least privilege, contextual output encoding, safe templating, and suitable Content Security Policy as defense in depth.

Experiment 28 — Armitage: GUI workflow and common issues

Run in: Armitage/Metasploit on Kali, if compatible; isolated vulnerable VM only.

Use an isolated vulnerable VM and take a snapshot first.

Confirm the installed Armitage/Metasploit versions; legacy Armitage may not support the current framework.

Start only the database/framework services required by the installed release.

Launch Armitage using the method documented by the installed package.

Connect to the local Metasploit instance.

Add or scan only the assigned lab host, using the instructor-approved discovery method.

Review discovered hosts/services and record the evidence.

If the instructor explicitly requires an exploit demonstration, choose only the approved module and confirm all options target the isolated lab VM.

Record the attack path and observed impact; stop the session and restore the snapshot.

Do not use public targets or run exploit modules outside the authorized lab.

Experiment 29 — Robtex: workflow and cross-checks

Run in: Robtex website in a browser; Kali terminal for DNS/WHOIS cross-checks.

Open the official Robtex website in a browser.

Search the assigned domain or IP.

Review available DNS records, related domains/IPs, and historical information.

Record the source page and date.

Cross-check with:

```
dig <DOMAIN> A
dig <DOMAIN> NS
dig <DOMAIN> MX
dig -x <TARGET_IP>
whois <TARGET_IP>
```

Explain uncertainty: shared hosting, CDNs, cloud providers, and historical records can link unrelated organizations.

Experiment 30 — pfSense: detailed firewall practical

Run in: pfSense webConfigurator GUI; test client commands run on the specified Windows/Kali client.

Safe setup and rules

Log in to the pfSense webConfigurator using the lab credentials.

Open **Status** → **Interfaces** and identify WAN, LAN, and any lab interface addresses.

Open **Firewall** → **Rules** → **LAN** (or the interface named in the question).

Click **Add**. Choose the required action, protocol, source, destination, destination port, and description.

For an allow rule, restrict source/destination/port to the minimum needed. For a block test, use the assigned lab client and a known test service.

Click **Save** → **Apply Changes**.

Check **Status** → **System Logs** → **Firewall** and confirm whether the test traffic was allowed or blocked.

Test one allowed service and one intentionally blocked service from the assigned client.

Keep management access available; do not change the management rule without a recovery route.

Client verification

```
Test-NetConnection <LAB_SERVER_IP> -Port 80
Test-NetConnection <LAB_SERVER_IP> -Port 443
Test-NetConnection <LAB_SERVER_IP> -Port 22
```

or on Kali:

```
nc -vz <LAB_SERVER_IP> 80
nc -vz <LAB_SERVER_IP> 443
nc -vz <LAB_SERVER_IP> 22
```

A failed connection can be caused by a stopped service as well as a firewall rule. Verify the server is listening before concluding that the firewall blocked it.

Experiment 31 — Snort: rules, alerts, and PCAP workflow

Run in: Kali VM terminal; check Snort major version before using Snort 2-style commands.

Version and configuration

```
snort -V
snort -h
snort -T -c <SNORT_CONF> -i <LAB_INTERFACE>
```

The `-T`, `-c`, `-i`, `-r`, and alert options shown in this handbook are common Snort 2 patterns; Snort 3 uses a different configuration/option model in several areas. Check `snort -h` and the installed package documentation.

Review rules and logs

```
grep -n 'include' <SNORT_CONF>
grep -n 'alert ' <RULE_FILE>
find /var/log -maxdepth 3 -iname '*snort*' -type f 2>/dev/null
```

GUI/editor workflow for a local lab rule

Open the Snort rules file specified by the active configuration using an editor with administrator privileges.

Review the rule include path in the active configuration.

Add or edit only the instructor-provided benign lab rule.

Test the configuration before starting/restarting the sensor.

Generate only the instructor-approved test traffic or analyze the supplied PCAP.

Inspect console/log alerts and record timestamps, source/destination, rule message, and SID.

Remove temporary rules after the lab and retest the configuration.

Windows quick-launch and troubleshooting reference

Open common Windows tools

- **PowerShell as Administrator:** Start → type `PowerShell` → right-click → Run as administrator.
- **Command Prompt:** Start → type `cmd` → Enter.
- **Event Viewer:** Win+R → `eventvwr.msc`.

- **Services:** Win+R → `services.msc`.
- **Device Manager:** Win+R → `devmgmt.msc`.
- **Network Connections:** Win+R → `ncpa.cpl`.
- **Task Manager:** Ctrl+Shift+Esc.
- **Windows Firewall advanced rules:** Win+R → `wf.msc`.
- **Certificate store (current user):** Win+R → `certmgr.msc`.
- **System information:** Win+R → `msinfo32`.

Windows network troubleshooting

```
ipconfig /all
ipconfig /flushdns
Resolve-DnsName example.com
Test-NetConnection example.com -Port 443
Get-NetAdapter
Get-NetIPConfiguration
Get-NetRoute -AddressFamily IPv4
Get-NetNeighbor -AddressFamily IPv4
Get-NetTCPConnection -State Listen
```

Use `ipconfig /flushdns` only when clearing the local resolver cache is relevant to the practical.

Evidence checklist for any experiment

- [ ] Correct experiment number and assigned target scope
- [ ] Tool name and version
- [ ] Exact command or GUI menu path used
- [ ] Date/time and target identifier
- [ ] Raw output, screenshot, capture, or exported report
- [ ] Findings separated from assumptions
- [ ] Severity/impact and practical remediation
- [ ] Limitations, errors, and any failed steps documented
- [ ] Lab changes cleaned up or VM snapshot restored

Final environment check before the exam

This document separates the intended execution environment, but commands have **not** been executed against your individual Kali VM or Windows installation. Before exam day:

Run each tool's `--version` / `-h` / `--help` check where available.

For GUI tools, open the application once and confirm it reaches its main screen.

Confirm Kali can reach only the assigned lab network and that the Windows/Kali applications are installed on the intended machine.

Test commands against your instructor-provided lab target, not the documentation example IPs.

If a command's option is rejected, use the installed version's help rather than assuming every online example matches your version.